COLLEGIUM WITELONA
Uczelnia Państwowa

Wydział Nauk Technicznych i Ekonomicznych

Kierunek Informatyka

Specjalność Programowanie Aplikacji Mobilnych i Internetowych

Sebastian Górski

Wieloplatformowy system monitorowania urządzeń sieciowych z modułem wykrywania anomalii opartym na AI

Praca dyplomowa inżynierska
napisana pod kierunkiem
dr inż. Zbigniew Fryźlewicz

Legnica, 2026 rok

Spis treści

Wstęp **3**

1 Założenia projektowe **5**

1.1 Wprowadzenie 5

1.2 Przedmiot pracy i wizja aplikacji 5

1.3 Wymagania funkcjonalne 6

1.4 Wymagania niefunkcjonalne 8

1.5 Zarys architektury 9

1.5.1 Architektura logiczna 9

1.5.2 Architektura fizyczna 11

2 Warstwa API **13**

2.1 Struktura bazy danych 13

Wstęp

Sieci komputerowe stanowią obecnie podstawę funkcjonowania przedsiębiorstw, instytucji, a także gospodarstw domowych. Oprócz komputerów i serwerów pracuje w nich coraz więcej urządzeń, które do prawidłowego działania wymagają stałego połączenia z siecią, w tym urządzeń Internetu Rzeczy (ang. *Internet of Things*, IoT). Międzynarodowy Związek Telekomunikacyjny (ITU) definiuje Internet Rzeczy jako globalną infrastrukturę, która umożliwia świadczenie zaawansowanych usług dzięki łączeniu ze sobą przedmiotów fizycznych i wirtualnych z wykorzystaniem technologii informacyjno-komunikacyjnych [www1]. W praktyce urządzeniami IoT są m.in. czujniki, kamery, inteligentne liczniki czy sterowniki maszyn, które samodzielnie wymieniają dane przez sieć. Według szacunków firmy IoT Analytics liczba podłączonych urządzeń IoT na świecie miała na koniec 2025 roku osiągnąć 21,1 miliarda, a w 2030 roku może wynieść około 39 miliardów [www2]. Wraz ze wzrostem liczby urządzeń rośnie znaczenie ich nadzoru. Utrata łączności z serwerem, przełącznikiem lub urządzeniem sterującym pracą maszyn może prowadzić do przestojów i strat, dlatego awarię należy wykryć jak najszybciej, najlepiej zanim zgłoszą ją użytkownicy. Monitorowanie urządzeń w sieci lokalnej utrudnia jednak fakt, że zazwyczaj nie są one dostępne z zewnątrz, a ich stan można sprawdzić wyłącznie z wnętrza tej sieci.

Celem pracy jest zaprojektowanie i implementacja systemu NetHelt, umożliwiającego monitorowanie dostępności oraz jakości połączenia urządzeń działających w sieciach lokalnych (LAN) z poziomu przeglądarki internetowej. System ma pozwolić użytkownikowi na nadzorowanie wielu sieci w jednym miejscu, bez konieczności udostępniania urządzeń poza sieć lokalną, oraz na szybkie otrzymywanie informacji o wykrytych problemach. Celem dodatkowym jest sprawdzenie, czy algorytm `Isolation Forest` pozwala na wczesne wykrywanie pogorszenia jakości połączenia, zanim dojdzie do całkowitej utraty łączności.

Zakres pracy obejmuje analizę wymagań, projekt architektury systemu, implementację jego modułów oraz wdrożenie w chmurze AWS. W ramach pracy zrealizowano aplikację serwerową w języku Java 25 z wykorzystaniem frameworka Spring Boot, bazy danych PostgreSQL oraz pamięci podręcznej Redis, a także aplikację webową w frameworku Angular 21. Klient desktopowy, instalowany w monitorowanej sieci jako usługa systemowa dla systemów Windows, Linux i macOS, pobiera z serwera zadania monitorujące, wykonuje je (ping, sprawdzenie portu TCP, kontrola odpowiedzi HTTP) i odsyła wyniki, buforując je w przypadku braku połączenia. Aplikacja umożliwia logowanie, również za pośrednictwem kont Google i GitHub, wykrywanie

urządzeń w sieci lokalnej oraz definiowanie reguł alertów, o których spełnieniu użytkownik jest powiadamiany pocztą elektroniczną, przez komunikator Telegram lub w aplikacji. Administrator ma dostęp do zarządzania kontami i rolami użytkowników, statystyk aktywności, logów błędów oraz ręcznego uruchamiania zadań harmonogramu. Zakres obejmuje również opracowanie i przetestowanie modelu `Isolation Forest` na statystykach czasu odpowiedzi, agregowanych w jednominutowych oknach czasowych, oraz udostępnienie go jako bezserwerowej usługi AWS Lambda, z którą komunikuje się aplikacja serwerowa. Analizę AI użytkownik włącza dla wybranych zadań, a wykryte anomalie są zgłaszane w formie powiadomień. Poza zakresem pracy pozostaje aplikacja mobilna oraz graficzny interfejs klienta desktopowego.

Praca składa się z TODO: liczba rozdziałów. Rozdział 1 przedstawia założenia projektowe systemu: wizję aplikacji, wymagania funkcjonalne i niefunkcjonalne oraz architekturę logiczną i fizyczną. Rozdział 2 opisuje warstwę API, w tym strukturę bazy danych. Rozdział **??** dotyczy modelu sztucznej inteligencji do wykrywania anomalii, czyli doboru cech wejściowych, treningu oraz testów różnych konfiguracji modelu. TODO: opis kolejnych rozdziałów: klient desktopowy, aplikacja webowa, testy. Pracę zamyka podsumowanie, zawierające ocenę stopnia realizacji celu oraz kierunki dalszego rozwoju systemu.

1. Założenia projektowe

1.1 Wprowadzenie

W niniejszym rozdziale przedstawiono założenia projektowe systemu NetHelt, opracowanego w ramach pracy inżynierskiej. Rozdział obejmuje opis przedmiotu pracy, wizję tworzonej aplikacji, wymagania funkcjonalne i niefunkcjonalne oraz zarys architektury logicznej i fizycznej systemu.

Celem przedstawionych założeń jest określenie sposobu realizacji systemu monitorowania urządzeń sieciowych działających w środowisku lokalnym. Opisane zostały zarówno wymagania dotyczące funkcjonalności aplikacji, jak również aspekty związane z bezpieczeństwem, wydajnością oraz komunikacją pomiędzy poszczególnymi modułami systemu.

Projektowany system ma charakter wielomodułowy i składa się z aplikacji webowej, aplikacji desktopowej działającej jako klient monitorujący oraz modułu analizy danych wykorzystującego algorytmy sztucznej inteligencji do wykrywania anomalii sieciowych. W rozdziale przedstawiono również ogólną koncepcję architektury rozwiązania oraz relacje pomiędzy jego komponentami.

Założenia opisane w dalszej części pracy stanowią podstawę do implementacji systemu oraz realizacji poszczególnych etapów projektu.

1.2 Przedmiot pracy i wizja aplikacji

Przedmiotem pracy jest opracowanie wielomodułowego systemu informatycznego do monitorowania urządzeń sieciowych działających w środowisku lokalnym (LAN). System umożliwia kontrolę stanu urządzeń, analizę jakości połączenia sieciowego oraz wykrywanie anomalii mogących świadczyć o problemach w działaniu infrastruktury sieciowej.

Projektowane rozwiązanie składa się z aplikacji webowej, aplikacji desktopowej oraz modułu analizy danych wykorzystującego algorytm `Isolation Forest`. Aplikacja webowa odpowiada za zarządzanie systemem i prezentację danych użytkownikowi, natomiast aplikacja desktopowa realizuje operacje monitorujące w sieci lokalnej i przesyła wyniki do centralnego serwera.

System przewiduje obsługę użytkowników standardowych oraz administratorów, posiadających rozszerzone uprawnienia zarządzania systemem. Aplikacja umożliwia m.in. konfigurację urządzeń, monitorowanie ich dostępności, analizę parametrów połączenia oraz wysyłanie powiadomień o wykrytych problemach.

Wizja projektu zakłada stworzenie skalowalnego i łatwego w rozbudowie systemu, dostępnego z poziomu przeglądarki internetowej oraz wdrożonego w środowisku chmurowym.

Komunikacja pomiędzy modułami jest realizowana z wykorzystaniem interfejsów API oraz centralnej bazy danych PostgreSQL.

1.3 Wymagania funkcjonalne

Wymagania funkcjonalne określają funkcje oraz operacje, które system powinien realizować z perspektywy użytkownika. Ich celem jest opisanie zachowania aplikacji oraz możliwości udostępnianych poszczególnym aktorom systemu.

W procesie analizy wymagań wykorzystano koncepcję *user stories*, stosowaną w metodykach zwinnych do opisu oczekiwań użytkowników wobec systemu [www3]. Na podstawie przeprowadzonej analizy opracowano zestaw wymagań funkcjonalnych dla projektowanej aplikacji.

W systemie wyróżniono trzech głównych aktorów:

- administratora,
- użytkownika,
- gościa.

Administrator posiada uprawnienia do zarządzania użytkownikami oraz konfiguracją systemu. Użytkownik może zarządzać własnymi urządzeniami i konfiguracją monitorowania. Gość jest niezalogowanym użytkownikiem aplikacji i posiada dostęp wyłącznie do podstawowych funkcji systemu.

Zestaw wymagań funkcjonalnych przedstawiono w tabeli 1.1.

Tabela 1.1: Wymagania funkcjonalne systemu

ID	User Story
F-G1	Jako gość, chcę zobaczyć informację na temat aplikacji, aby zrozumieć jej przeznaczenie
F-G2	Jako gość, chcę się zarejestrować, aby uzyskać dostęp do sytemu
F-G3	Jako gość, chcę się zalogować, aby korzystać z funkcji systemu
F-G4	Jako gość, chcę zresetować hasło, aby odzyskać dostęp do konta
F-U1	Jako użytkownik, chcę dodać urządzenie, aby je monitorować
F-U2	Jako użytkownik, chcę przeglądać listę swoich urządzeń, aby mieć wgląd w ich stan
F-U3	Jako użytkownik, chcę usunąć urządzenie, aby utrzymać porządek
F-U4	Jako użytkownik, chcę edytować dane urządzenia, aby zachować aktualność informacji
F-U5	Jako użytkownik, chcę konfigurować powiadomienia, aby otrzymywać informację na bieżąco
F-U6	Jako użytkownik, chcę przeglądać statystyki moich urządzeń, aby w łatwy sposób kontrolować czasy odpowiedzi
F-U7	Jako użytkownik, chcę pobrać klienta desktopowego, aby skonfigurować połączenie w mojej sieci
F-U8	Jako użytkownik, chcę zmienić swoje hasło, aby zachować bezpieczeństwo konta
F-U9	Jako użytkownik, chcę się wylogować, aby zablokować nieautoryzowany dostęp do mojego konta
F-A1	Jako administrator, chcę przeglądać listę użytkowników, aby mieć wgląd w konta w systemie
F-A2	Jako administrator, chcę usuwać i przywracać konta, aby zapewnić bezpieczeństwo systemu
F-A3	Jako administrator, chcę zmieniać role użytkowników, aby kontrolować ich uprawnienia
F-A4	Jako administrator, chcę resetować hasła użytkowników, aby umożliwić odzyskanie dostępu
F-A5	Jako administrator, chcę ręcznie wywoływać zadania z harmonogramu, aby mieć większą kontrolę nad przebiegiem działania aplikacji
F-A6	Jako administrator, chcę przeglądać ilość aktywnych użytkowników w danym okresie, aby być świadomym poziomu sukcesu aplikacji

Źródło: opracowanie własne.

1.4 Wymagania niefunkcjonalne

Wymagania niefunkcjonalne określają ograniczenia oraz cechy jakościowe systemu informatycznego. W przeciwieństwie do wymagań funkcjonalnych nie opisują one konkretnych funkcji aplikacji, lecz definiują sposób działania systemu, jego wydajność, bezpieczeństwo, niezawodność oraz dostępność [www4].

Wymagania niefunkcjonalne mają istotny wpływ na jakość projektowanego rozwiązania oraz komfort korzystania z systemu przez użytkowników. Ich analiza została przeprowadzona na podstawie literatury z zakresu inżynierii oprogramowania oraz założeń projektowych aplikacji [1].

Najważniejsze wymagania niefunkcjonalne projektowanego systemu przedstawiono w tabeli 1.2.

Tabela 1.2: Wymagania niefunkcjonalne systemu

Kategoria	Wymaganie
Wydajność	System powinien obsługiwać minimum 100 równoczesnych użytkowników
Wydajność	Czas odpowiedzi API powinien być krótszy niż 500 ms dla 95% zapytań
Wydajność	System powinien wykorzystywać mechanizm cache'owania najczęściej używanych danych w celu redukcji czasu odpowiedzi o co najmniej 30%
Wydajność	Moduł AI powinien przetwarzać jednominutowe okno danych pojedynczego urządzenia w czasie krótszym niż 100 ms
Dostępność	System powinien być dostępny przez minimum 99% czasu w skali miesiąca
Dostępność	Maksymalny jednorazowy czas niedostępności systemu nie powinien przekraczać 1 godziny
Bezpieczeństwo	System powinien wykorzystywać autoryzację użytkownika opartą o JWT oraz OAuth2
Bezpieczeństwo	Komunikacja pomiędzy klientem a serwerem powinna być szyfrowana z wykorzystaniem protokołu HTTPS
Bezpieczeństwo	Hasła użytkowników powinny być przechowywane w postaci skrótu kryptograficznego z wykorzystaniem algorytmu BCrypt
Niezawodność	System powinien zapisywać logi błędów oraz umożliwiać ich przeglądanie administratorowi

Źródło: opracowanie własne.

1.5 Zarys architektury

Architektura systemu została zaprojektowana w sposób warstwowy, zgodnie z podejściem opisywanym przez R.C. Martina, które zakłada separację logiki biznesowej od szczegółów implementacyjnych oraz infrastruktury systemowej [2]. Takie podejście umożliwia utrzymanie wysokiej spójności systemu oraz ułatwia jego dalszy rozwój i testowanie.

1.5.1 Architektura logiczna

Architektura logiczna opisuje strukturę systemu z perspektywy jego komponentów funkcjonalnych oraz relacji między nimi, niezależnie od środowiska uruchomieniowego. Koncentruje się na podziale odpowiedzialności oraz przepływie danych pomiędzy modułami.

System został podzielony na następujące elementy logiczne:

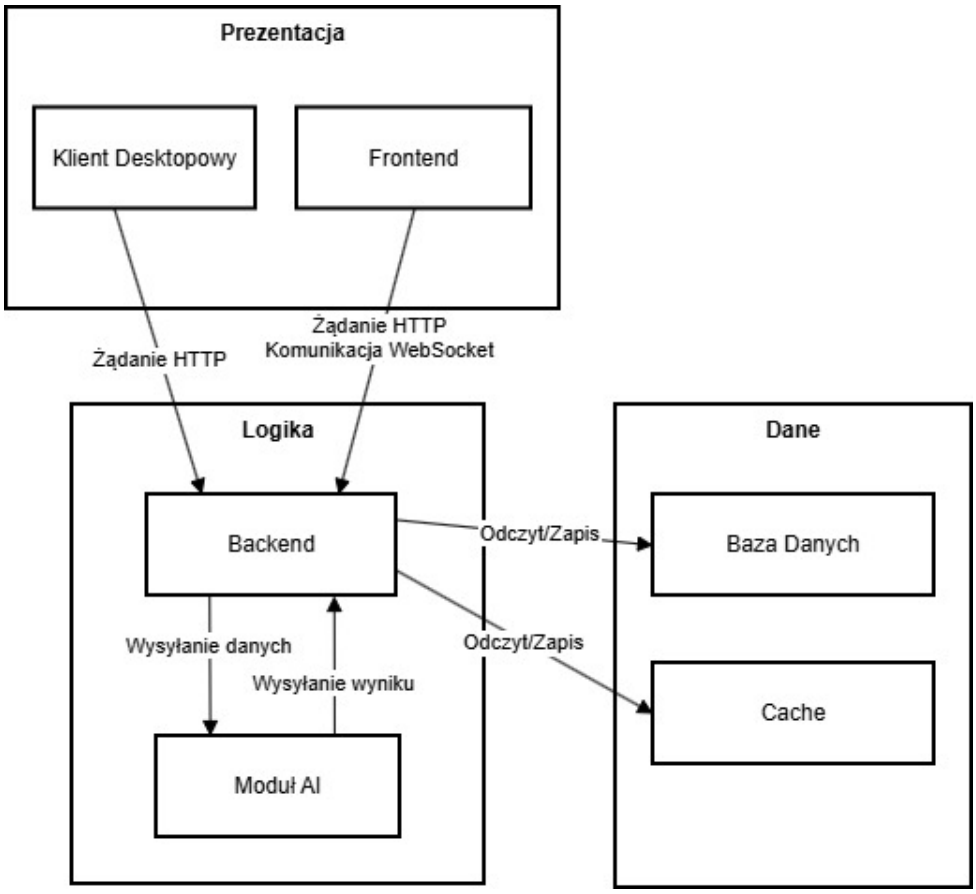
- Frontend (aplikacja webowa),
- Backend (REST API + WebSocket),
- Moduł AI,
- Baza danych,
- Cache,
- Klient desktopowy.

Elementy te zostały zorganizowane w trzy warstwy:

- warstwa prezentacji,
- warstwa logiki aplikacyjnej,
- warstwa danych.

Warstwa prezentacji obejmuje frontend oraz klienta desktopowego, które odpowiadają za interakcję z użytkownikiem. Warstwa logiki obejmuje backend oraz moduł AI, które realizują przetwarzanie żądań, logikę biznesową oraz analizę danych. Warstwa danych obejmuje bazę danych oraz mechanizm cache, odpowiedzialne za przechowywanie danych w sposób trwały oraz tymczasowy.

Komunikacja pomiędzy komponentami przebiega w następujący sposób: frontend komunikuje się z backendem poprzez REST API oraz WebSocket, zapewniając komunikację w czasie rzeczywistym, natomiast klient desktopowy wykorzystuje REST API. Backend przetwarza żądania i deleguje zadania do modułu AI w celu analizy danych, a następnie zapisuje lub odczytuje dane z warstwy danych.



Rysunek 1.1: Architektura logiczna systemu
Źródło: opracowanie własne

1.5.2 Architektura fizyczna

Architektura fizyczna przedstawia sposób wdrożenia systemu w środowisku infrastrukturalnym. System został wdrożony na platformie Amazon Web Services (AWS). Przy doborze usług kierowano się przede wszystkim niskim kosztem utrzymania, dlatego zamiast droższych usług zarządzanych wykorzystano jedną maszynę wirtualną, na której uruchomiono większość komponentów serwerowych.

Komponenty serwerowe zostały skonteneryzowane przy użyciu technologii Docker. Każdy z nich działa w osobnym kontenerze na instancji usługi Amazon EC2, a kontenery są zarządzane za pomocą narzędzia Docker Compose. Zapewnia to ich izolację oraz możliwość niezależnego wdrażania.

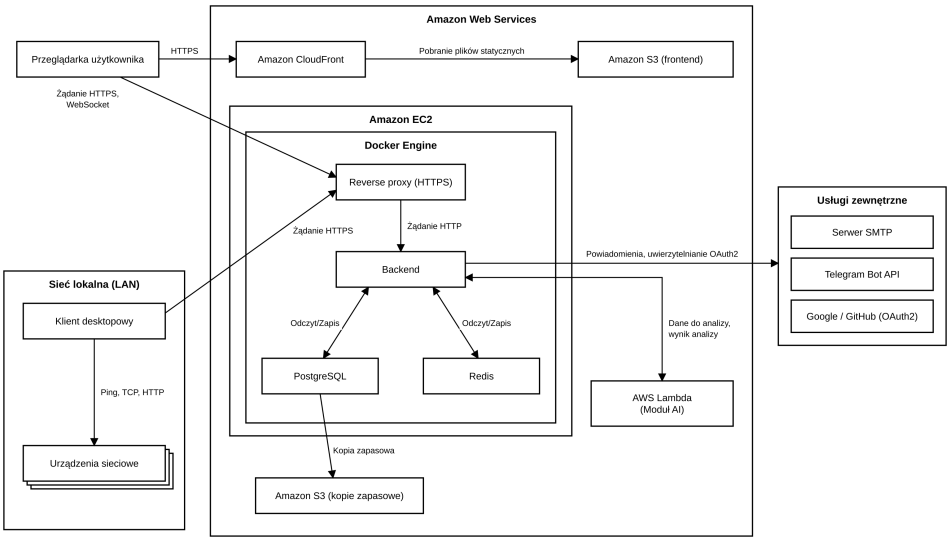
W warstwie fizycznej system obejmuje:

- backend uruchamiany w kontenerze Docker na instancji EC2,
- bazę danych PostgreSQL uruchamianą w kontenerze Docker na instancji EC2, której kopia zapasowa jest codziennie zapisywana w usłudze Amazon S3,
- cache Redis uruchamiany w kontenerze Docker na instancji EC2,

- serwer pośredniczący (*reverse proxy*), zapewniający szyfrowanie HTTPS w komunikacji z backendem,
- frontend w postaci plików statycznych przechowywanych w usłudze Amazon S3 i udostępnianych przez sieć CDN Amazon CloudFront,
- moduł AI uruchamiany w usłudze bezserwerowej AWS Lambda,
- klienta desktopowego instalowanego lokalnie na urządzeniu użytkownika.

Backend korzysta ponadto z usług zewnętrznych: serwera SMTP do wysyłki wiadomości e-mail, interfejsu Telegram Bot API do wysyłki powiadomień przez komunikator Telegram oraz usług Google i GitHub do uwierzytelniania użytkowników w protokole OAuth2. Schemat wdrożenia przedstawiono na rysunku 1.2.

Takie podejście zapewnia separację komponentów oraz niski koszt utrzymania, a także upraszcza proces wdrażania. Moduł AI działa w modelu bezserwerowym, więc generuje koszty wyłącznie w czasie wykonywania analizy. Wadą rozwiązania jest umieszczenie komponentów serwerowych na jednej maszynie wirtualnej. W przypadku wzrostu liczby użytkowników bazę danych oraz cache można przenieść do usług zarządzanych Amazon RDS i Amazon ElastiCache bez zmian w kodzie aplikacji.



Rysunek 1.2: Architektura fizyczna systemu
Źródło: opracowanie własne

Spis rysunków

1.1 Architektura logiczna systemu Źródło: opracowanie własne 11
1.2 Architektura fizyczna systemu Źródło: opracowanie własne 12

2.1 Diagram bazy danych aplikacji NetHelt Źródło: Opracowanie własne w witrynie internetowej https://dbdiagram.io/ 14

Spis tabel

1.1 Wymagania funkcjonalne systemu 7
1.2 Wymagania niefunkcjonalne systemu 9

Bibliografia

Wydawnictwa zwarte

[1] I. Sommerville, *Inżynieria oprogramowania*, wyd. 10, Wydawnictwo Naukowe PWN, Warszawa, 2016.

[2] R.C. Martin, *Czysta architektura. Struktura i design oprogramowania. Przewodnik dla profesjonalistów*, Wydawnictwo Helion, Gliwice, 2018.

Netografia

[www1] ITU-T, *Recommendation ITU-T Y.2060 (06/2012): Overview of the Internet of things*, International Telecommunication Union, Genewa, 2012, `https://www.itu.int/rec/T-REC-Y.2060-201206-I` [dostęp: 08.11.2026].

[www2] S. Sinha, *State of IoT 2025: Number of connected IoT devices growing 14% to 21.1 billion globally*, IoT Analytics, 2025, `https://iot-analytics.com/number-connected-iot-devices/` [dostęp: 08.11.2026].

[www3] Atlassian, *User stories*, `https://www.atlassian.com/pl/agile/project-management/user-stories` [dostęp: 25.05.2026].

[www4] Perforce, *What Are Non-Functional Requirements?*, `https://www.perforce.com/blog/alm/what-are-non-functional-requirements-examples` [dostęp: 25.05.2026].